

数值计算方法 第四次上机作业

自动化（控制）

1 问题描述

考虑区间 $[-1, 1]$ 上的函数：

$$f(x) = \frac{1}{1 + 100x^2}$$

以 $x_i = -1 + \frac{2i}{n}$, $i = 0, 1, 2, \dots, n$ 为插值节点，取不同的分点数目 $n = 2, 4, 6, 8, 10$ ，分别进行 Lagrange 插值、分段线性插值，并画出原函数 $f(x)$ 及插值多项式函数在 $[-1, 1]$ 上的图像。

2 问题分析

本题要求对函数 $f(x) = \frac{1}{1 + 100x^2}$ 在等距节点上分别进行 Lagrange 插值和分段线性插值，并通过图像对比观察两种插值方法的效果差异。

Lagrange 插值使用全部 $n + 1$ 个节点构造一个次数不超过 n 的多项式，使其经过所有给定的数据点。当节点数较少时，插值多项式能较好地逼近原函数；但随着节点数增多，等距节点上的高次多项式插值在区间两端会出现剧烈振荡，即 Runge 现象。

分段线性插值则采用不同的策略：在每两个相邻节点之间用一条直线段连接，形成折线函数。这种方法虽然整体光滑性不如多项式插值，但不会出现振荡现象，并且随着节点数的增加，折线会越来越贴近原函数。

3 Lagrange 插值

3.1 算法分析

给定 $n + 1$ 个插值节点 $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ ，Lagrange 插值多项式为：

$$L_n(x) = \sum_{i=0}^n y_i \cdot l_i(x)$$

其中 $l_i(x)$ 为 Lagrange 基函数：

$$l_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

每个基函数满足 $l_i(x_k) = \delta_{ik}$ ，即在自身节点处取值为 1，在其他节点处取值为 0。程序实现时，对每个待求点 x ，通过三层循环（遍历绘图点、求和、求积）计算插值结果。

3.2 MATLAB 程序

完整代码附在报告最后，此处仅展示 Lagrange 插值函数的核心代码。

```
function y_plot = lagrange_interp(x_nodes, y_nodes, x_plot)
% Lagrange 插值函数
    n = length(x_nodes);
    m = length(x_plot);
    y_plot = zeros(1, m);
    for k = 1:m % 遍历每个绘图点
        for i = 1:n % 求和
            li = 1;
            for j = 1:n % 求积: 计算基函数 l_i(x)
                if j ~= i
                    li = li * (x_plot(k) - x_nodes(j)) ...
                        / (x_nodes(i) - x_nodes(j));
                end
            end
            y_plot(k) = y_plot(k) + y_nodes(i) * li;
        end
    end
end
```

3.3 运行结果及分析

Lagrange 插值与原函数的对比图如图 1 所示。

从图中可以观察到以下现象：

当 $n = 2$ 时，插值多项式为二次函数，虽然经过三个节点，但整体形状与原函数相差较大。随着 n 的增大，插值多项式在区间中部对原函数的逼近效果有所改善，但在区间两端出现了越来越剧烈的振荡。特别是当 $n = 10$ 时，Lagrange 插值多项式在 x 接近 ± 1 处的值已经达到了 ± 5 左右，远远偏离了原函数值，这就是典型的 Runge 现象。

Runge 现象产生的根本原因在于：对于等距节点上的高次多项式插值，插值误差上界中包含的因子 $\prod_{i=0}^n (x - x_i)$ 在区间端点附近会变得很大，导致误差随多项式次数的增加而急剧增大，尤其是对于像 $f(x) = \frac{1}{1 + 100x^2}$ 这样在区间端点处导数变化剧烈的函数。

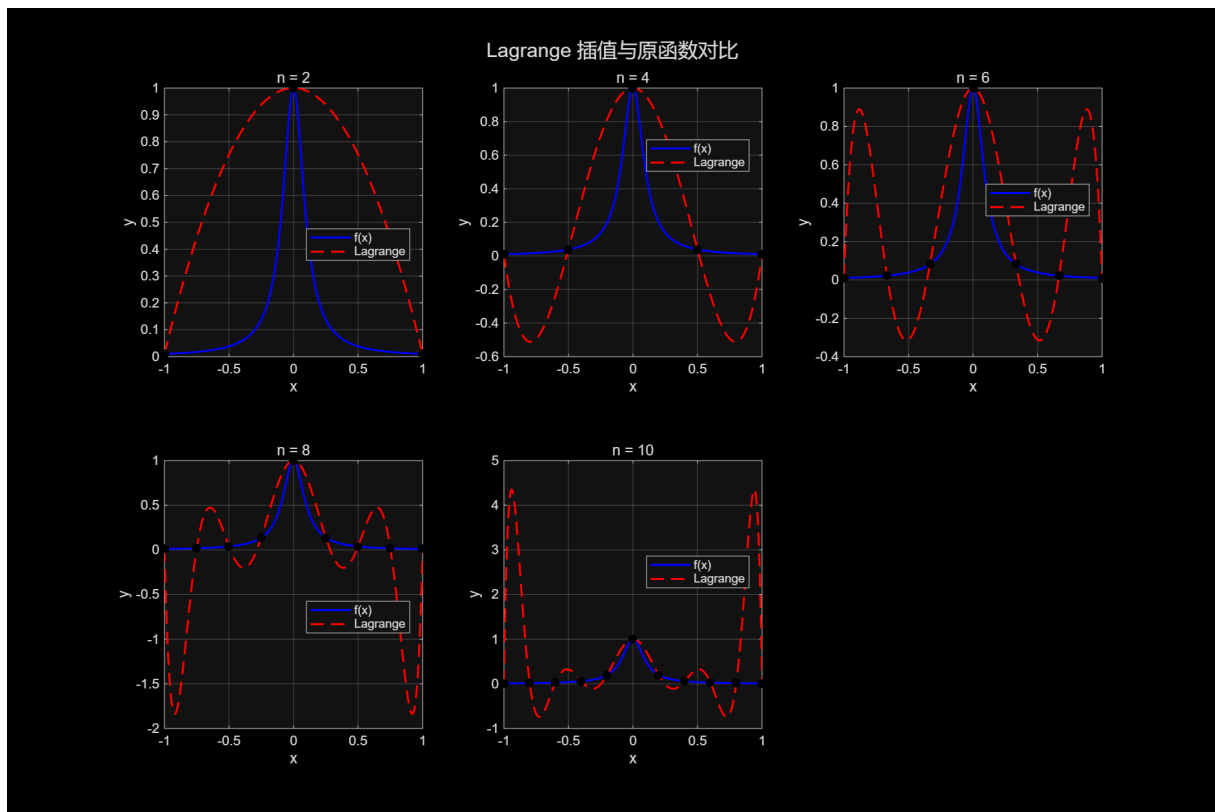


图 1: Lagrange 插值与原函数对比

4 分段线性插值

4.1 算法分析

分段线性插值的基本思想是：对于每个待求点 x ，先找到它所在的区间 $[x_i, x_{i+1}]$ ，然后在该区间内用通过 (x_i, y_i) 和 (x_{i+1}, y_{i+1}) 的直线进行插值：

$$P(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i), \quad x \in [x_i, x_{i+1}]$$

分段线性插值的优点是简单稳定，不会出现高次多项式插值的振荡现象。其误差与步长 $h = x_{i+1} - x_i$ 及函数的二阶导数有关，当节点数增多时，步长减小，插值精度相应提高。

4.2 MATLAB 程序

完整代码附在报告最后，此处仅展示分段线性插值函数的核心代码。

```
function y_plot = piecewise_linear_interp(x_nodes, y_nodes, x_plot)
% 分段线性插值函数
m = length(x_plot);
n = length(x_nodes);
y_plot = zeros(1, m);
for k = 1:m
```

```

x = x_plot(k);
% 找到 x 所在的区间
if x <= x_nodes(1)
    idx = 1;
elseif x >= x_nodes(n)
    idx = n - 1;
else
    for i = 1:n-1
        if x >= x_nodes(i) && x <= x_nodes(i+1)
            idx = i;
            break;
        end
    end
end
% 线性插值
x0 = x_nodes(idx);    x1 = x_nodes(idx + 1);
y0 = y_nodes(idx);    y1 = y_nodes(idx + 1);
y_plot(k) = y0 + (y1 - y0) * (x - x0) / (x1 - x0);
end
end

```

4.3 运行结果及分析

分段线性插值与原函数的对比图如图 2 所示。

从图中可以观察到：分段线性插值不会出现 Lagrange 插值中的端点振荡现象。当 $n = 2$ 时，折线与原函数偏差较大，但随着 n 增加到 8、10，折线逐渐贴近原函数的形状。

另外值得注意的一个细节是：由于题目选取的分段数 n （2、4、6、8、10）均为偶数，因此 $x = 0$ 这个点恰好始终是一个插值节点。所以在这几个偶数等分下，原函数的最高峰 $f(0) = 1$ 均被折线精准命中，没有误差；分段线性插值的误差主要体现在尖峰两侧（如 $x = \pm 0.1$ 附近）被拉平的地方。总体而言，分段线性插值的逼近效果随节点数的增加而稳定改善，没有任何发散的迹象。

5 两种方法比较

从本次实验可以看出，对于 $f(x) = \frac{1}{1 + 100x^2}$ 这类在区间端点处导数变化剧烈的函数，使用等距节点进行高次 Lagrange 多项式插值是不合适的——随着插值次数增加，端点处的振荡误差反而越来越大。分段线性插值虽然光滑性不如多项式插值，但具有稳定收敛的优势，不会出现 Runge 现象。

在实际应用中，若需要兼顾光滑性和稳定性，可以考虑使用三次样条插值（既保证 C^2 连续，又避免高次多项式的振荡），或者采用 Chebyshev 节点代替等距节点来进行多

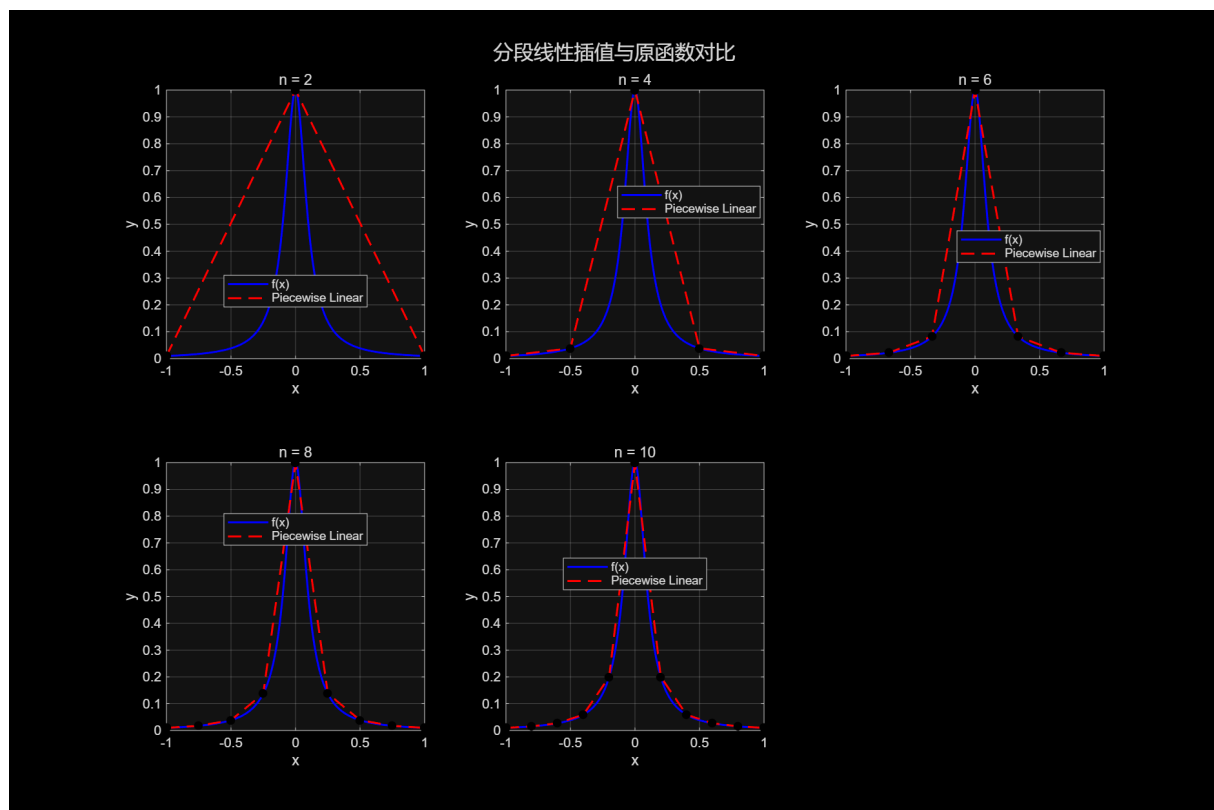


图 2: 分段线性插值与原函数对比

比较维度	Lagrange 插值	分段线性插值
插值函数类型	全局高次多项式	分段一次多项式
光滑性	C^∞ (无穷次可微)	C^0 (连续但不光滑)
Runge 现象	有, 端点振荡严重	无
n 增大时的表现	端点误差急剧增大	整体误差稳定减小
计算复杂度	$O(n^2)$	$O(n)$

表 1: Lagrange 插值与分段线性插值的比较

项式插值，从而有效抑制 Runge 现象。

6 心得体会

通过这次上机实验，我完整体验并观察到了 Runge 现象的发生过程：随着等距节点 n 的增加，Lagrange 插值多项式在区间端点处的振荡愈发剧烈， $n = 10$ 时边缘的插值结果已经跑飞了。与之相对，分段线性插值虽然看起来有棱有角，但在增加节点时能稳稳地收敛于原函数图像。

另外，在写 Lagrange 插值的代码时，我直接用了三层 for 循环（遍历绘图点、求和、求积）。虽然对付 $n \leq 10$ 这种规模瞬秒出图，但如果有成百上千个数据点， $O(n^2)$ 的复杂度其实很不划算。总体而言，这次实验让我明白：在数值计算里，暴力拉高精度（插值次高）往往行不通，选择合适的算法结构才是最重要的。

7 附：完整 MATLAB 程序

```
clear; clc; close all;

% 定义原函数
f = @(x) 1 ./ (1 + 100 * x.^2);

% 绘图用的密集点
x_plot = linspace(-1, 1, 500);
y_exact = f(x_plot);

% 不同的分点数目
n_values = [2, 4, 6, 8, 10];

%% Lagrange 插值
figure('Name', 'Lagrange Interpolation', 'Position', [100, 100, 1200, 800]);
for idx = 1:length(n_values)
    n = n_values(idx);

    % 生成等距插值节点
    x_nodes = -1 + 2*(0:n)/n;
    y_nodes = f(x_nodes);

    % 计算 Lagrange 插值
    y_lagrange = lagrange_interp(x_nodes, y_nodes, x_plot);

    % 绘图
    subplot(2, 3, idx);
```

```

    plot(x_plot, y_exact, 'b-', 'LineWidth', 1.5); hold on;
    plot(x_plot, y_lagrange, 'r--', 'LineWidth', 1.5);
    plot(x_nodes, y_nodes, 'ko', 'MarkerSize', 6, 'MarkerFaceColor', 'k');
    xlabel('x'); ylabel('y');
    title(['n = ', num2str(n)]);
    legend('f(x)', 'Lagrange', 'Location', 'best');
    grid on;
    hold off;
end
sgtitle('Lagrange 插值与原函数对比');
saveas(gcf, 'lagrange_interp.png');

%% 分段线性插值
figure('Name', 'Piecewise Linear Interpolation', 'Position', [100, 100, 1200, 800]);
for idx = 1:length(n_values)
    n = n_values(idx);

    % 生成等距插值节点
    x_nodes = -1 + 2*(0:n)/n;
    y_nodes = f(x_nodes);

    % 计算分段线性插值
    y_piecewise = piecewise_linear_interp(x_nodes, y_nodes, x_plot);

    % 绘图
    subplot(2, 3, idx);
    plot(x_plot, y_exact, 'b-', 'LineWidth', 1.5); hold on;
    plot(x_plot, y_piecewise, 'r--', 'LineWidth', 1.5);
    plot(x_nodes, y_nodes, 'ko', 'MarkerSize', 6, 'MarkerFaceColor', 'k');
    xlabel('x'); ylabel('y');
    title(['n = ', num2str(n)]);
    legend('f(x)', 'Piecewise Linear', 'Location', 'best');
    grid on;
    hold off;
end
sgtitle('分段线性插值与原函数对比');
saveas(gcf, 'piecewise_linear_interp.png');

fprintf('绘图完成，图片已保存。\\n');

%% ===== 函数定义 =====

function y_plot = lagrange_interp(x_nodes, y_nodes, x_plot)
% Lagrange 插值函数
    n = length(x_nodes);

```

```

    m = length(x_plot);
    y_plot = zeros(1, m);

    for k = 1:m % 遍历每个绘图点
        for i = 1:n % 求和
            li = 1;
            for j = 1:n % 求积：计算基函数 l_i(x)
                if j ~= i
                    li = li * (x_plot(k) - x_nodes(j)) / (x_nodes(i) - x_nodes(j));
                end
            end
            y_plot(k) = y_plot(k) + y_nodes(i) * li;
        end
    end
end

function y_plot = piecewise_linear_interp(x_nodes, y_nodes, x_plot)
% 分段线性插值函数
    m = length(x_plot);
    n = length(x_nodes);
    y_plot = zeros(1, m);

    for k = 1:m
        x = x_plot(k);
        % 找到 x 所在的区间
        if x <= x_nodes(1)
            idx = 1;
        elseif x >= x_nodes(n)
            idx = n - 1;
        else
            for i = 1:n-1
                if x >= x_nodes(i) && x <= x_nodes(i+1)
                    idx = i;
                    break;
                end
            end
        end
        % 线性插值
        x0 = x_nodes(idx);
        x1 = x_nodes(idx + 1);
        y0 = y_nodes(idx);
        y1 = y_nodes(idx + 1);
        y_plot(k) = y0 + (y1 - y0) * (x - x0) / (x1 - x0);
    end
end
end

```